Knowledge Base: Submission Brief

Repository: github.com/sarthakagrawal927/knowledge-base

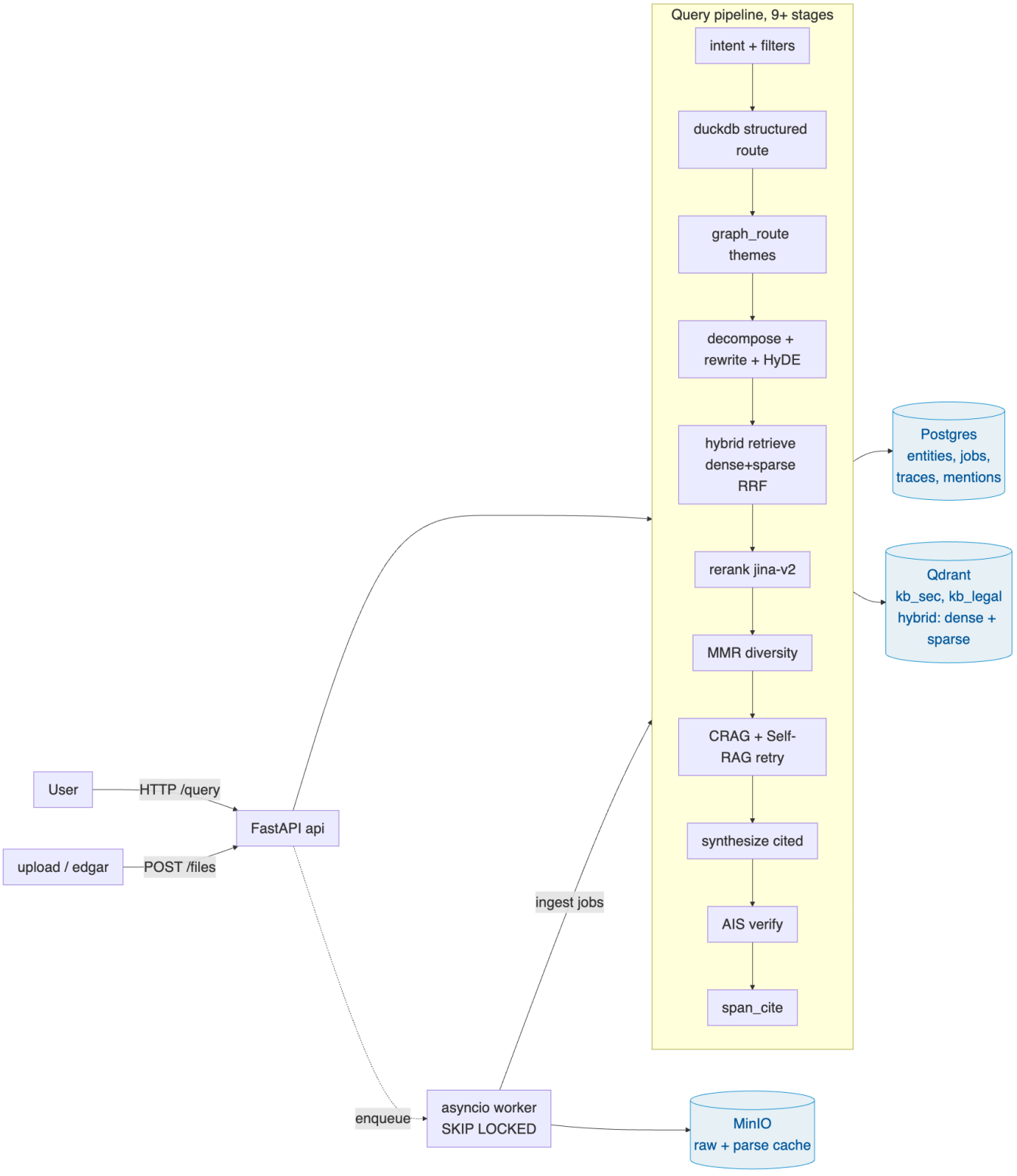
Short brief. The depth lives in the repo:

- NOTES.md: decision log with research links.
- DESIGN.md: architecture detail and the boundary tests for the domain-agnostic claim.
- README.md: bootstrap, endpoints, reading guide.

What I built supports schema-defined domain onboarding: drop a YAML, drop in files, ask questions, get cited answers. The schema layer (`src/kb/schema`) and the vector store (`src/kb/vector`) carry no domain identifiers. Grep confirms zero hits for SEC, Legal, ticker, FinancialMetric, RiskFactor, or Clause across either directory. The two-domain demo (SEC EDGAR + SPDX legal licenses) runs the schema-swap path on the same code.

What it isn't, end-to-end, is fully domain-neutral. Four places in `src/kb/query` and `src/kb/extract` still carry SEC-flavoured defaults: `graph_route.py` branches on `domain == "sec"` for its default entity type, the intent classifier's few-shot examples are all SEC-flavoured (Apple, NVDA, EPS-Diluted), the DuckDB route has a `TICKER_FROM_FILENAME` regex baked in, and `xlsx_bridge.py` is explicitly financial-metric specific. The right shape is for each to live in `domains/<name>/config.yaml`; queued in §3. The Legal demo proves the schema-swap path (schema, retrieval, citations), but mostly bypasses those domain-flavoured stages because its questions don't trigger them.

1. Architecture



Three stores, each with one job. Postgres holds the versioned schemas, the entities the pipeline extracts (lineage walked via recursive CTEs), and the ingest job queue. `SELECT ... FOR UPDATE SKIP LOCKED` makes that a real queue without bringing in Celery. Qdrant holds the chunks, with native dense + sparse hybrid and RRF fusion; a pgvector adapter is shipped behind the same Protocol so the choice isn't permanent. MinIO holds raw bytes and cached parse artifacts. Workers fan out as an asyncio pool, with per-file failures bounded so one bad PDF doesn't poison the rest of the index.

2. The three trickiest decisions

Parse once, re-extract many. The cache sits at the Unstructured `Element` boundary, keyed on `sha256(file_bytes)`. Elements preserve bbox and page provenance, so OCR and `hi_res` layout detection don't have to re-run when a schema changes. Schema edits create a new `ingest_jobs` row keyed on `(file_id, schema_id)`, and the parse-cache hit makes re-extract substantially cheaper than re-parsing/OCR (the LLM extraction call still runs, but the expensive parse step is skipped). This is what makes the rubric's "schema change shouldn't redo expensive parsing" requirement cheap to satisfy.

Layered retrieval. Hybrid retrieval (dense vector search plus a sparse keyword-style search, fused via RRF) catches both paraphrase and rare tokens. A cross-encoder rerank gives a precision score over the top candidates. For aggregate questions ("which companies had revenue over $60B?"), the engine generates SQL over the entities table in DuckDB instead of going through retrieval. Per-claim verification checks that the cited source actually supports each claim. Each layer catches a different failure class; the layered evidence is what drives the final confidence signal.

Citation as a first-class invariant. Every retrieval path (hybrid, structured DuckDB, theme-based, low-confidence retry) converges on the same triple, `(file_id, page, excerpt)`. Confidence is downgraded proportionally to per-claim verify pass rate, so "the model said it" and "the citation re-verified" carry different confidence values rather than the same one.

3. What I'd do differently with more time

In rough priority order: real graph storage replacing the current theme-routing sketch (community detection on the entity co-mention graph would give themes a reusable structure across queries); a larger natural-question eval set per domain to push variance below the noise floor; migrating the SEC-flavoured defaults in `query/` and `extract/` out to per-domain config (per the qualifier above); per-token SSE streaming on `/query` (endpoint exists, currently emits stage-level events only); a small set of committed seed fixtures so `make seed` works offline.

4. Where it breaks today

- **Without an LLM key, structured extraction yields zero entities.** Chunks still ship to Qdrant so retrieval keeps working, but the schema-driven outputs and the DuckDB aggregate route both go quiet.
- **Cross-file entity merging is one-pass.** A duplicate arriving later under a slightly different display name creates a near-duplicate canonical. A nightly reconciliation pass would fix it; not shipped.
- **Citations are page + best-sentence-accurate, not character-accurate.** True character-range highlighting in the original PDF would need a sentence-back-to-bbox mapping.
- **Worker memory isn't bounded.** Concurrency is count-based; `hi_res` on a large 10-K across 4 workers can OOM a small VM. The right fix is a memory-aware semaphore per stage.
- **SEC seed depends on live EDGAR.** `make seed` pulls 10 10-K filings live; on failure it falls back to XLSX-only (`src/kb/seed/sec_seed.py:116`). Legal seed reads committed fixtures from `domains/legal/fixtures/` first, so `make seed-legal` runs offline. SEC fixtures aren't committed because each filing is several MB; queued in §3.

The remaining rubric items (schema versioning with NL field descriptions, identity merging, lineage, idempotency, scoped + filtered + conversational retrieval, layered configurability) are represented in the implementation under `src/kb/`; the source-tree map in README walks through where each one lives.